

LEGALEASE

Set Up the Development Environment

Professional Project Documentation — 20 Pages

Project Scope

LegalEase is a proposed legal-information and document-assistance application. This document explains how to establish a safe, reproducible development environment for the project and connects the environment setup to ideation, requirements, design, planning, development, testing, documentation, demonstration, diagrams, and visual assets.

Prepared as an academic/project implementation guide

Technology focus: Python • FastAPI • Virtual Environment • REST API • SQLite/PostgreSQL-ready • Git • Automated Testing

Note: LegalEase is a software project concept. It should not be treated as a substitute for advice from a qualified legal professional.

PAGE 2 — Brainstorming and Ideation

LegalEase begins with a practical problem: users often struggle to organize legal information, understand document requirements, and locate the right workflow. The project therefore focuses on structured information delivery rather than presenting itself as a lawyer or making unsupported legal conclusions.

Core idea: build a modular application in which the user interface communicates with a backend API. The backend validates requests, applies application rules, retrieves stored content, and returns structured responses.

Initial brainstorming themes

- Legal document checklist generator based on user-selected categories.
- Plain-language explanations of common legal terminology.
- Document metadata and version tracking.
- Search and filtering for stored legal resources.
- Secure user authentication as a future module.
- Audit logging for important application actions.
- Role-based access for administrator and normal users.
- API-first architecture so web and mobile clients can reuse the backend.
- Clear separation between software functionality and legal advice.
- Automated tests to reduce regressions as features grow.

Ideation output: a small, testable MVP should be implemented first. The development environment must support quick iteration, reproducible dependencies, API testing, source control, and clear documentation.

PAGE 3 — Requirement Analysis

Requirement analysis translates the idea into measurable system behavior. For the environment setup, the most important requirement is reproducibility: another developer should be able to clone the project, create the same environment, install the declared dependencies, run the server, and execute the tests.

Functional requirements

- The project shall expose backend endpoints through FastAPI.
- The project shall validate request data using typed schemas.
- The project shall provide a health/status endpoint for environment verification.
- The project shall separate application code from configuration.
- The project shall support automated unit/API testing.
- The project shall store dependencies in a requirements file or modern Python project configuration.
- The project shall use Git for source-code version control.
- The project shall document setup, execution, testing, and troubleshooting.

Non-functional requirements

- Readable and maintainable code.
- Repeatable local setup.
- Reasonable security defaults.
- Clear error handling.
- Fast startup for local development.
- Extensible architecture for database and authentication modules.
- No hard-coded passwords, API keys, or private credentials.

Acceptance criteria: a fresh machine can create the virtual environment, install dependencies, start LegalEase, open the API documentation, and run the test suite without modifying source code.

PAGE 4 — Project Design

The recommended design is a layered backend. The API layer receives HTTP requests; schemas define the shape of data; service logic contains application rules; repository/database code handles persistence; configuration manages environment-specific settings.

Suggested folder structure

```
LegalEase/  
  app/  
  main.py  
  core/  
  config.py  
  api/  
  routes.py  
  schemas/  
  legal.py  
  services/  
  legal_service.py  
  repositories/  
  legal_repository.py  
  tests/  
  test_health.py  
  .env.example  
  .gitignore  
  requirements.txt  
  README.md
```

This structure prevents the main application file from becoming a single large script. It also allows later additions such as database migrations, authentication, logging, and an administrative dashboard without redesigning the entire project.



Figure 1. High-level LegalEase backend architecture.

Design principle: keep responsibilities separate. The environment should make these boundaries easy to test independently.

PAGE 5 — Project Planning

A staged plan reduces risk. The development environment is established before feature development so that every later milestone has the same foundation.

Suggested milestone plan

Phase	Work	Output
1	Ideation	Problem statement and scope
2	Requirements	Functional/non-functional requirements
3	Environment	Python, venv, dependencies, Git
4	Backend skeleton	FastAPI app and routes
5	Testing	Automated test suite
6	Documentation	README and API guide
7	Demonstration	Recorded/live workflow

Risk controls

- Pin or constrain dependency versions where appropriate.
- Use a virtual environment rather than system-wide packages.
- Keep secrets outside Git.
- Run tests before merging changes.
- Document known limitations and assumptions.
- Use small commits so errors can be isolated.

PAGE 6 — Development Environment Setup

The development environment is the central focus of this document. A clean environment makes the project easier to reproduce and reduces the chance of dependency conflicts.

Recommended tools

- Python 3.11+ for a modern, well-supported development baseline.
- Visual Studio Code, PyCharm, or another Python-capable editor.
- Git for version control.
- A terminal such as PowerShell, Windows Terminal, Bash, or zsh.
- A virtual environment created specifically for LegalEase.
- An API client such as the browser-based FastAPI documentation or a REST client.
- Optional: SQLite for the first local database stage and PostgreSQL for a production-oriented stage.

Verify Python and Git

```
python --version
git --version
```

Create the project directory and virtual environment

```
mkdir LegalEase
cd LegalEase
python -m venv .venv
```

Activation differs by operating system. On Windows PowerShell use **.venv\Scripts\Activate.ps1**. On macOS/Linux use **source .venv/bin/activate**. The terminal should then indicate that the virtual environment is active.

PAGE 7 — Dependency Installation and Configuration

Once the virtual environment is active, update packaging tools and install the core backend dependencies. A simple academic project can begin with FastAPI, Uvicorn, and pytest.

Install packages

```
python -m pip install --upgrade pip
pip install fastapi uvicorn[standard] pytest httpx
```

Freeze a reproducible dependency list

```
pip freeze > requirements.txt
```

For a team project, review the generated list and avoid adding unrelated packages. A modern alternative is to manage dependencies in a `pyproject.toml` file. The important principle is that the dependency declaration should be committed to source control so another developer can recreate the environment.

Recommended .gitignore entries

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
.env
.vscode/
.idea/
*.sqlite3
```

Security note

Never commit real passwords, access tokens, private keys, or production connection strings. Commit an `.env.example` file containing variable names and safe placeholder values instead.

PAGE 8 — FastAPI Application Creation

The first executable LegalEase backend can be intentionally small. This proves that the environment is correct before the project adds database, authentication, document processing, or search functionality.

app/main.py

```
from fastapi import FastAPI

app = FastAPI(
    title="LegalEase API",
    version="1.0.0",
    description="Backend API for the LegalEase project."
)

@app.get("/")
def root():
    return {"message": "LegalEase API is running"}

@app.get("/health")
def health():
    return {"status": "healthy"}
```

Start the development server

```
uvicorn app.main:app --reload
```

The `--reload` option is useful during development because the server restarts when source files change. It should not be treated as a production deployment configuration.

FastAPI automatically exposes interactive API documentation, making it possible to inspect and test endpoints from a browser. The health endpoint is especially useful because it provides a simple environment check.



Figure 2. First environment-verification request.

PAGE 9 — Configuration and Environment Variables

Configuration should be separated from application logic. Development, testing, and production can then use different values without rewriting source code.

.env.example

```
APP_NAME=LegalEase
APP_ENV=development
DEBUG=true
DATABASE_URL=sqlite:///./legalease.db
SECRET_KEY=replace_this_in_real_deployment
```

Configuration goals

- Use safe defaults for local development.
- Keep credentials outside the repository.
- Use different settings for testing and production.
- Validate required configuration at application startup.
- Document each variable and its purpose.
- Do not print secrets in logs.

A configuration module can later use Pydantic Settings or another configuration approach. For the first setup milestone, keeping the environment variable names documented is enough to establish a reliable convention.

Suggested environments

Development is for local coding and debugging. Testing is isolated and designed for repeatable automated checks. Production is a separately secured environment with controlled secrets, logging, database backups, and deployment procedures.

PAGE 10 — Development Workflow

A disciplined workflow keeps environment setup, coding, and testing connected. Each feature should be implemented in a small change, tested, documented, and committed.

Git initialization

```
git init
git add .
git commit -m "Initial LegalEase development environment"
```

Recommended daily cycle

- Pull or update the latest project changes.
- Activate the LegalEase virtual environment.
- Install updated dependencies when requirements change.
- Create a small feature or bug-fix branch.
- Implement the smallest useful change.
- Run formatting/linting tools if configured.
- Run automated tests.
- Start the API and perform a manual smoke test.
- Update documentation.
- Commit the completed change with a clear message.

Example branch names

```
feature/health-endpoint
feature/legal-document-search
fix/request-validation
docs/setup-guide
```

This workflow makes it easier to identify which change introduced a problem and keeps the main branch in a usable state.

PAGE 11 — Testing Strategy

Testing confirms that the environment and application behave as expected. LegalEase should use several levels of testing rather than relying only on manual demonstration.

Test levels

- Unit tests: verify individual functions or services.
- API tests: send requests to FastAPI routes and inspect responses.
- Integration tests: verify interactions between application layers and a test database.
- Smoke tests: confirm the server starts and key endpoints respond.
- Regression tests: preserve previously fixed behavior.
- Security-oriented tests: check input validation, authorization boundaries, and secret handling.

Example test

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "healthy"
```

Run tests

```
pytest -q
```

The test suite should be executed after environment changes and before a demonstration. Automated tests provide repeatable evidence that the basic development environment is functioning.

PAGE 12 — Testing Cases and Expected Results

The following test cases provide a practical checklist for the initial LegalEase environment.

ID	Test	Expected result
TC-01	Python version check	Supported Python version is reported.
TC-02	Virtual environment activation	Terminal uses the project environment.
TC-03	Dependency installation	Required packages install without unresolved errors.
TC-04	Server startup	Uvicorn starts the FastAPI application.
TC-05	Root endpoint	HTTP 200 and JSON message are returned.
TC-06	Health endpoint	HTTP 200 and healthy status are returned.
TC-07	Automated tests	pytest completes successfully.
TC-08	Git status	Unwanted secrets and environment folders are ignored.
TC-09	Configuration	Safe local values are loaded correctly.
TC-10	Documentation	A new developer can follow setup instructions.

Testing evidence should be recorded in the project log or README for academic evaluation. Screenshots can be added later as project-specific evidence.

PAGE 13 — Documentation Plan

Documentation is part of development, not a final afterthought. LegalEase should explain what the system does, how to install it, how to run it, how to test it, and what limitations apply.

README structure

```
# LegalEase
## Project Overview
## Features
## Requirements
## Development Environment
## Installation
## Configuration
## Running the API
## API Endpoints
## Testing
## Project Structure
## Troubleshooting
## Limitations
## Future Enhancements
```

Developer documentation should include

- Supported Python version.
- Exact installation commands.
- Environment activation commands for the target operating systems.
- Dependency installation instructions.
- How to launch the API.
- Where interactive API documentation is available.
- How to execute tests.
- How to reset a local database if one is introduced.
- Known limitations and unresolved issues.
- Security and privacy notes.

For a legal-information application, documentation should clearly state the boundary between informational software features and professional legal advice. Content should be reviewed for jurisdiction and currency before being used in a real deployment.

PAGE 14 — Demonstration Procedure

A successful demonstration should prove the environment works from start to finish. The presenter should avoid skipping directly to screenshots because the objective is to show a reproducible workflow.

Recommended demonstration sequence

- Show the LegalEase project folder.
- Show the virtual environment and requirements declaration.
- Open the main application file.
- Activate the environment in the terminal.
- Start Uvicorn with reload enabled.
- Open the API documentation.
- Execute the root endpoint.
- Execute the health endpoint.
- Run pytest and show the result.
- Show Git status and the documented project structure.
- Explain one planned feature and how it will be added safely.
- End with limitations and future scope.



Figure 3. Demonstration workflow.

Demonstration evidence

Useful evidence includes terminal screenshots, API documentation screenshots, a successful health response, pytest output, and the project tree. Avoid placing real secrets or personal data in screenshots.

PAGE 15 — System Diagrams

Diagrams make the architecture easier to explain in a project review or viva. The following diagrams can be recreated in draw.io, PowerPoint, Figma, or a diagramming library for a polished presentation.

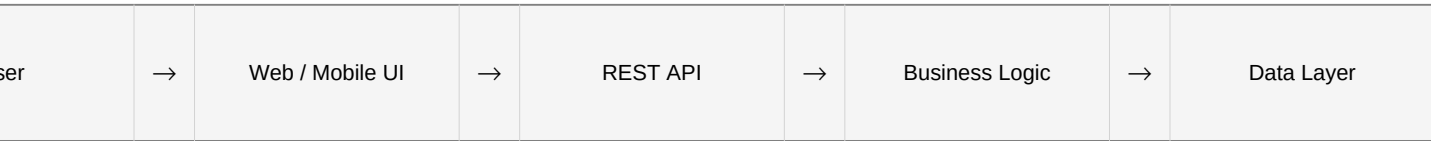


Figure 4. LegalEase request flow.



Figure 5. Backend processing pipeline.

Component responsibilities

- User interface: collects input and presents results.
- API routes: define HTTP endpoints and status codes.
- Schemas: validate and serialize structured data.
- Services: contain reusable business/application logic.
- Repositories: isolate data access.
- Database: stores persistent records.
- Tests: verify behavior across layers.

PAGE 16 — Images and Visual Documentation

A professional project report should contain purposeful visuals. Instead of adding decorative images only, each visual should prove a concept or document an implementation step.

Recommended project images

- Image 1: LegalEase project folder structure in the editor.
- Image 2: Terminal showing virtual environment activation.
- Image 3: Dependency installation and requirements file.
- Image 4: FastAPI code in the editor.
- Image 5: Running Uvicorn server.
- Image 6: Interactive API documentation.
- Image 7: Health endpoint JSON response.
- Image 8: pytest success output.
- Image 9: Architecture diagram.
- Image 10: Final demonstration screen.

Image-generation prompt for a project illustration

Create a clean professional educational infographic titled “LegalEase Development Environment”. Show a laptop with a Python FastAPI code editor, a terminal with a virtual environment, API documentation, automated testing, Git version control, and a simple legal-document icon. Use a modern blue/white academic technology style, readable labels, balanced spacing, and a clean vector-like presentation.

When inserting screenshots into a final report, crop unnecessary personal information and ensure credentials, tokens, and private files are not visible.

PAGE 17 — Troubleshooting and Risk Management

Environment setup can fail for predictable reasons. The project should include a troubleshooting section so that common problems do not stop development.

Problem	Likely cause	Action
python not found	Python/PATH issue	Install Python or correct PATH; reopen terminal.
pip install fails	Network/package/build issue	Check network, Python version, and package error message.
ModuleNotFoundError	Wrong environment	Activate .venv and reinstall requirements.
Port already in use	Another process owns the port	Stop the old process or run Uvicorn on another port.
pytest import error	Package path/configuration	Run from project root and check package structure.
Secret appears in Git	Missing .gitignore	Remove secret, rotate it if real, and update ignore rules.

Risk-management principle: never solve an environment problem by disabling security controls blindly. Identify the exact error, isolate the cause, make the smallest safe change, and record the solution.

PAGE 18 — Development Roadmap and Future Enhancements

After the environment milestone is stable, LegalEase can grow in controlled stages. The environment should remain unchanged as much as possible while features are added behind clear modules and tests.

Phase 1 — MVP

- Health/status API.
- Legal resource categories.
- Search endpoint.
- Basic document metadata.
- Input validation.
- Unit and API tests.

Phase 2 — Data and users

- Database integration.
- User registration and authentication.
- Role-based access control.
- Audit logs.
- Database migrations.

Phase 3 — Advanced features

- Document upload with controlled file types.
- Full-text search.
- Versioned legal-resource content.
- Notifications and workflow tracking.
- Administrative content review.
- Deployment automation and monitoring.

Any feature involving legal conclusions, personal legal records, or sensitive documents requires additional privacy, security, jurisdiction, content-review, and professional-legal oversight before real-world use.

PAGE 19 — Implementation Checklist and Deliverables

The following checklist can be used as the final verification sheet before project submission or demonstration.

- ■ Problem statement and project scope are documented.
- ■ Brainstorming and ideation outputs are recorded.
- ■ Functional requirements are listed.
- ■ Non-functional requirements are listed.
- ■ Project architecture is documented.
- ■ Python version is recorded.
- ■ Virtual environment is created.
- ■ Dependencies are declared.
- ■ .gitignore is configured.
- ■ Environment variables are documented safely.
- ■ FastAPI application starts successfully.
- ■ Root endpoint works.
- ■ Health endpoint works.
- ■ Interactive API documentation works.
- ■ Automated tests pass.
- ■ Project structure is documented.
- ■ Troubleshooting notes are available.
- ■ Architecture diagrams are included.
- ■ Screenshots/images are prepared.
- ■ Demonstration steps are rehearsed.
- ■ Limitations are stated.
- ■ Future enhancements are listed.
- ■ No real secrets are included in the submission.
- ■ Final report is proofread and versioned.

Final deliverables

Source code • requirements.txt or pyproject.toml • README • test suite • diagrams • screenshots • demonstration script • project report • environment setup instructions.

PAGE 20 — Conclusion

Setting up the LegalEase development environment is the foundation for every later stage of the project. A carefully prepared Python virtual environment, controlled dependencies, FastAPI application skeleton, automated tests, Git workflow, configuration strategy, and documentation create a stable platform for future development.

The project should progress from ideation to requirements, design, planning, development, testing, documentation, and demonstration in an iterative cycle. Diagrams and screenshots make the technical work easier to communicate, while a structured troubleshooting process reduces development risk.

Final environment command sequence

```
mkdir LegalEase
cd LegalEase
python -m venv .venv

# Activate the environment
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate

python -m pip install --upgrade pip
pip install fastapi uvicorn[standard] pytest httpx
pip freeze > requirements.txt

# Create app/main.py, then:
uvicorn app.main:app --reload

# In another terminal:
pytest -q
```

Project success definition: a new developer can follow this report, reproduce the environment, start LegalEase, verify the API, run the tests, understand the architecture, and continue development without relying on undocumented personal knowledge.

Academic and legal-use note

This document is a software-development guide. LegalEase should provide informational and workflow-oriented functionality unless qualified legal professionals and appropriate governance establish a different scope. Legal rules change by jurisdiction and over time, so production legal content requires current source verification and professional review.

END OF 20-PAGE DOCUMENT